



Plug-and-Play Higher-Order Process Integration

Johannes Neubauer and Bernhard Steffen, *Technische Universität Dortmund, Germany*

Unexpected hurdles in dealing with integration, variability, and interoperability in software development can be overcome using higher-order process integration. Even in its simplicity-oriented, tamed version, this approach fosters a powerful plug-and-play discipline, where processes and services are moved around like data.

Computer applications are deeply integrated into our daily activities in a wide range of areas—from personal activities such as shopping, travel, banking, and paying bills, to business functions such as payroll, taxes, and inventory tracking. These activities are handled by different applications of widely varying quality and flexibility. Indeed, there are often different solutions for a single problem or task, even from the same provider, depending on the user profile or context. Variance occurs in the look and feel and the required user process, not to mention the API interactions between systems. As an unintended consequence, users struggle with wide technical diversity in the applications, technical developers struggle with increasingly challenging implementations, and businesses grapple with exploding maintenance costs.

In 2011, with the launch of the Business Process Model and Notation 2.0 (BPMN 2.0)¹—a standard for business process modeling that uses graphical representation to specify business processes—the area of BPM heated up with the promise that application-level business processes would now be directly executable. This advance was due to BPMN 2.0's potential to close the semantic gap—a key source of

misunderstanding between business (dealing with the purpose of the application) and IT (focused on the technical implementation)—and so BPM was seen as an almost-universal cure to diversity and variability problems. However, BPM's promise only bears out in specific scenarios and does not support variability because it lacks the means to manage process variation and variant-rich systems.²

Current BPM standards, such as the Business Process Execution Language (BPEL),³ BPMN,⁴ and even BPMN 2.0, are constrained to fixed bindings between business activities in a model and the services or substructures they refer to. Thus, each variant must be modeled explicitly via decision points that are modeling pendants—that is, activities that branch to different execution paths (successor activities)—to if- and switch-clauses in comprehensive but very large models; otherwise they are maintained individually as a collection of separate entities. Both alternatives are problematic: comprehensive modeling leads to unmanageably large models, and the maintenance and support of many structurally similar individual variant models is a nightmare.

This is a long way from the simplicity BPM promised. Enhancing BPM with a tamed version of higher-order process passing breaks the spell by supporting a flexible form of type-based process integration, which also enables variability modeling and captures variants with functionality that was unknown at the process's start.⁵ In practice, deployed and even running processes can be seamlessly enhanced with new functionality without touching their code base. In fact, based on the higher-order concepts, new services and component processes can be selected, modified, constructed, and then safely passed as if they were data.

Tamed refers to both strict enforcement of type correctness and the explicit exclusion of recursion. Controlling

variability means controlling the huge wealth of combinations, as described by Anna-Lena Lamprecht, Stefan Naujokat, and Ina Schaefer, elsewhere in this issue.⁶ Thus, at the (business) process modeling level in particular, variability concerns and computational aspects should be separated in favor of simplicity and understandability.

We illustrate the impact of our dynamic, type-safe, application-level service and process integration via the FlexToll system, a small case study in which services and processes are exchanged at runtime in a plug-and-play fashion.

THE FLEXTOLL SCENARIO

Toll road systems, particularly as they are set up in Europe, can be annoying because they not only require drivers to pay toll fees, but also (force drivers) to wait in line behind others to physically do so. Some automated toll-paying solutions eliminate this delay, but many are too specific and require dedicated setups that typically only work for a single toll service provider and very few payment and onboard systems.

As the variety of payment services and local toll providers is difficult to change, an ideal generic onboard system should automatically adapt to these services' specific setups and infrastructures without requiring manual payment interaction, let alone queuing. In fact, the complete toll process should work in three simple steps:

- Toll stations and approaching cars automatically connect and search for a matching configuration between toll service and payment system. The automatic connector synthesis technology from the Connect Project⁷ or a new standard developed for this purpose and enforced could be used to support this communication.
- The toll station stores the matching configuration together with the car's entry point, time, and other relevant data as the car proceeds. Automatic payment processing is guaranteed. Only in the case of outage or system failure would the car need to queue for an entry ticket.
- When the car leaves the toll area, the toll is billed to the chosen payment system according to the rules of the applied toll service.

Why is this solution beyond the scope of current (business) process modeling? Because no existing solution deals with the *combinatorial explosion*: for t toll services,

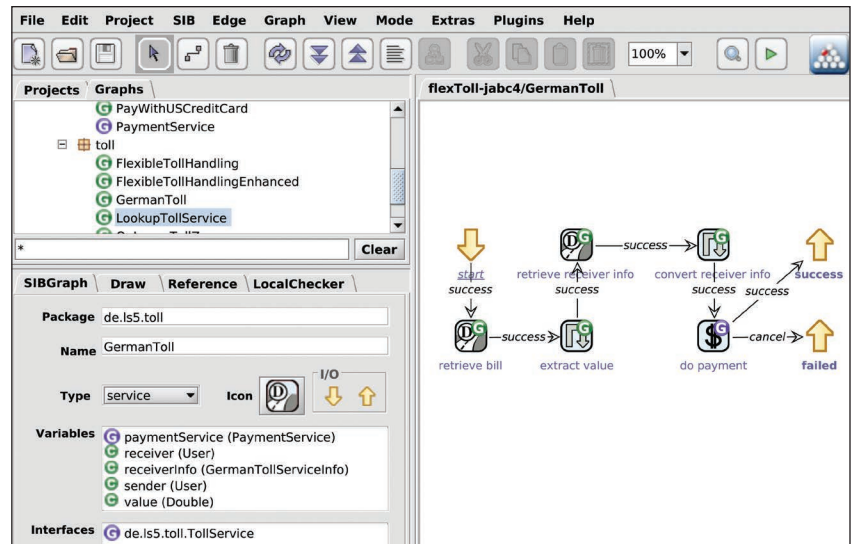


Figure 1. The eXtreme Model-Driven Development (XMDD) paradigm in action. The screenshot shows a collection of toll-related processes including *LookupTollService* and *GermanToll*.

p payment services, and c onboard systems, the fixed binding between business activities and services or the substructures they refer to implies that every combination must be specified, leading to an explosion in either quantity or size in the order of $O(t \times p \times c)$. Furthermore, current BPM solutions can only deal with entities known at design time. Adding a new entity requires a new release.

Relying on state-of-the-art product lines² and variation management⁸ might help simplify the required maintenance effort, but they do not reduce the combinatorial explosion or the need for new releases. Overcoming these limitations requires a mechanism and discipline that allow late discovery and binding of *entire processes*, and can construct ad hoc solutions in a simple way while still ensuring correctness. Taken together, higher-order thinking and the type safety concept from programming languages do the job.

HIGHER-ORDER EXTREME MODEL-DRIVEN DEVELOPMENT

We combine higher-order capabilities with the eXtreme Model-Driven Development paradigm.⁹ XMDD brings together ideas from service-oriented, model-driven design and the user-centeredness advocated in extreme programming. XMDD puts application experts in control of process design and evolution according to their level of technical competence and business responsibility. As an example, the Java Application Building Center (jABC)¹⁰ is an XMDD graphical framework for service-oriented design and development explicitly designed for nontechnical people.

Figure 1 shows the user interface for a subprocess in the FlexToll system. The *browser area* (Figure 1, top left) shows in tree form the projects, a collection of process

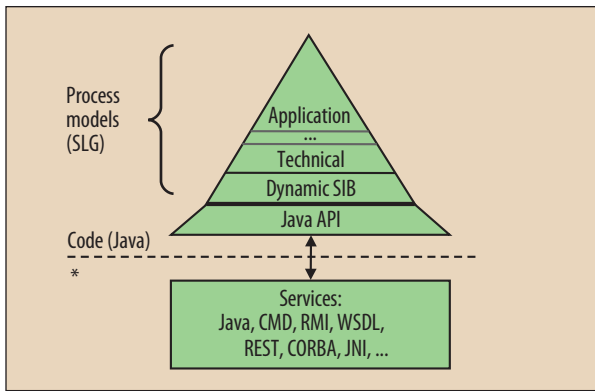


Figure 2. Dynamic pattern for business process modeling using Java as the base language. SIB, service-independent building block; SLG, service logic graph.

models (called service logic graphs, or SLGs) for the active project, and the library of available services or components. In the screenshot, no service browser tab is available at all, since—in the spirit of separation of concerns—they are only visible to specific (more technical) users. The collections of SLGs and services form the available domain of reusable, configurable processes and components.

In the *inspector area* (Figure 1, bottom left), several inspectors (SIBGraph, Draw, Reference, and LocalChecker) show internal information for the selected activity or SLG. In the snapshot, the SIBGraph inspector shows that *GermanToll* is a service and that it complies with the interface *TollService*. Also included in this window are its icon, variables, and variable types. The small blue G icon indicates that *paymentService* is a higher-order variable. This information can be edited. In the SIBGraph inspector, for example, the user can change the name of the graph; add, delete, or rename context variables (as well as set their type); and drag and drop variables to activities to wire data dependencies.

The *graph canvas* (Figure 1, right) is the modeling area, currently displaying the process model for the *GermanToll* service. Users build their process models by browsing the SLG and service libraries and dragging the icons of selected elements to the canvas, where they are instantiated as activities (SIBs, or service-independent building blocks). The user then connects them as appropriate, and, finally, they are configured via the inspectors.

The SLGs and their constituent activities (service-independent building blocks, or SIBs) are the only models used in application development. In fact, the one-thing approach (OTA)¹¹ provides the conceptual modeling infrastructure that enables all stakeholders (application experts, designers, component experts, implementers, quality assurers, and so on) to cooperate in an XMDD fashion.¹¹ In particular, OTA allows immediate user experience and seamless acceptance: all stakeholders

know, refine, and modify the same “thing” without duplications or the need to juggle different modeling languages or paradigms. Stakeholders can observe the progress of the development and the implications of decisions at their own level of expertise, which is a central trait of OTA.

Technically, SLGs are formal models with well-defined semantics in terms of Kripke transition systems (KTSs) that come with a wealth of corresponding analysis and synthesis tools. KTSs are generalizations of Kripke structures and labeled transition systems that allow labels on both nodes (the SIBs) and edges (the outgoing branches).¹² Operationally, the process models are similar to control-flow graphs: the SIBs represent activities and the branches describe how to continue the execution depending on the result of the previous activity.

SIBs can represent a single functionality or an entire subgraph (that is, another SLG),¹³ thus serving as a macro that hides more detailed process models. SIBs are parameterizable and communicate resources via shared *execution contexts*, a hierarchical concept comprising executable process instances. Because this approach is higher order, it allows high reusability of components (see, for example, the article by Cristina Secleanu and Ivica Crnkovic¹⁴ in this issue) as well as of entire (sub-)models and processes within larger applications: (sub-)models and processes can be passed around just like data.¹⁵

TYPE CONSISTENCY IN THE HIGHER-ORDER CONTEXT

Our approach combines strict type matching at the process level with the higher-order process integration described previously, so the application level and corresponding interface descriptions of business activities use compatible data types. Type transformations at the process level remove type mismatches. The benefits of this approach include agility, because integrating new processes or changing existing processes does not imply changes to any other services as long as interface compatibility is warranted, and interoperability, because strict type matching at the process level can hide differences between systems behind interface descriptions.

This way, services can be introduced or exchanged with different internal interfaces at any time in a plug-and-play fashion. However, the simplicity and ease of change management require a well-thought-out handoff between the technical level, which deals with implementation issues and the various programming language types and concepts, and the business (user) level, which deals with the process logic and virtual concepts that must be appropriately coupled or decoupled to guarantee a smooth plug-and-play experience.

Figure 2 shows our solution, which extends business- and application-oriented modeling beyond the scope of

current practice in BPM. We base our implementation on an established programming platform (in this case Java) that defines the coding environment for the code-level integration of external services, and thus the implementation language of the SIBs and SLGs, but we explicitly support permeability of concepts and techniques of the two worlds that cooperate in the middle layers to deliver the code-level interoperability for agile process models.

An SLG that supports higher-order process integration and guarantees type compatibility consists of possibly hierarchical business activities within jABC's graphical development environment. Coarse-grained SLGs, classified as application-level graphs, are successively refined by more specialized and technical SLGs, which in turn are based on dynamic SIBs. Dynamic SIBs are based on Java method calls and domain-specific data structures also defined in Java.

Our system uses the available type information to check whether the use of services and subgraphs passed this way is correct with regard to the underlying API expected by the higher-order SIB (caller) and provided by the SLG passed as parameter (callee). This type-checking mechanism, which overlaps the technical and business worlds, cultures, and practices, narrows the semantic gap and moves the hard technological break behind a Java API, where only technical experts are involved. In this sense it supports the principle, "simple for the many, difficult for the few."¹⁶

The fact that prominent business process management frameworks, such as Java BPM (jbPM) versions 4 and 5, Activiti, AristaFlow, and jABC, are all based on Java indicates that it is a good choice for implementing such a framework. However, Java is only used at the tool/system level—for example, to support consistent typing and object-oriented features such as subtyping and parametric polymorphism. Individual processes might use a different implementation language. Moreover, these concepts are quite general and can be implemented in and for other typed languages, such as C++ or C#.

The services level in Figure 2 includes the wealth of service and component paradigms that can serve as execution-time implementations for SIBs. Java—as a largely platform-independent language—is a good choice here as well, providing adapters to a wealth of other languages. For example, Web Services Description Language (WSDL) and Representational State Transfer (REST) stubs can be generated from the corresponding service descriptions. Java also allows the integration of arbitrary code, assuming it is accessible through a standard Java API. Almost every technology or method can be wrapped into a Java method: the Java Runtime Environment (JRE) provides interpreters for scripting languages such as Groovy, Jython, and JRuby, and the Java Native Interface (JNI) eases access to platform-dependent functions—for example, via C or C++

code. In fact, even command-line tools are accessible via standard Java libraries.

MODELING FLEXTOLL AS A PLUG-AND-PLAY APPLICATION

The sketch at the top of Figure 3 shows how FlexToll should function from the user's perspective: the bar should automatically go up as the vehicle approaches the entrance and exit toll gates at speed. The handwritten notes below this sketch reflect the toll stations' viewpoint: at the entry barrier, check the compatibility of all the software processes for the approaching vehicle and store the checkpoint data; later, at the exit barrier, recognize the car, calculate the price due for this journey, and issue the appropriate bill to the appropriate billing agency.

Services can be introduced or exchanged with different internal interfaces at any time in a plug-and-play fashion.

The process model shows how the onboard system handles this scenario. At the top level, the main process is launched when the toll station at the start checkpoint sends an event to the approaching car (activity *start*). Upon launch, FlexToll looks for the "right" *payment service* and *toll services*. If these lookup activities execute successfully, the onboard FlexToll system is prepared for payment at the exit point and the car passes the gate. Otherwise (for example, the car's onboard system attempts to use an unsupported payment method), the system switches to manual operation, and the driver must pay at the toll station. When the car reaches the exit station, the station's toll system recognizes the car, retrieves the corresponding data, and bills FlexToll in the local currency. Upon receiving the bill, FlexToll invokes the business activity *pay toll*, which references the previously stored toll process with the previously stored payment process instance (from the context variable *payService*) as a parameter. If the payment operation fails, it exits to *assistance needed*.

This process looks simple, but it requires not only determining the driver's preferred payment service but also a toll process that is compatible with the payment and toll systems. To exploit the full flexibility, such a system requires higher-order processes. Most current toll collection systems in Europe use fixed payment services as well as toll systems, and require user registration and an onboard unit (OBU). Consequently, a truck driver who drives from Hamburg to Greece might have quite a collection of identities with different national and local toll collection organizations, and an impressive collection of OBU devices.

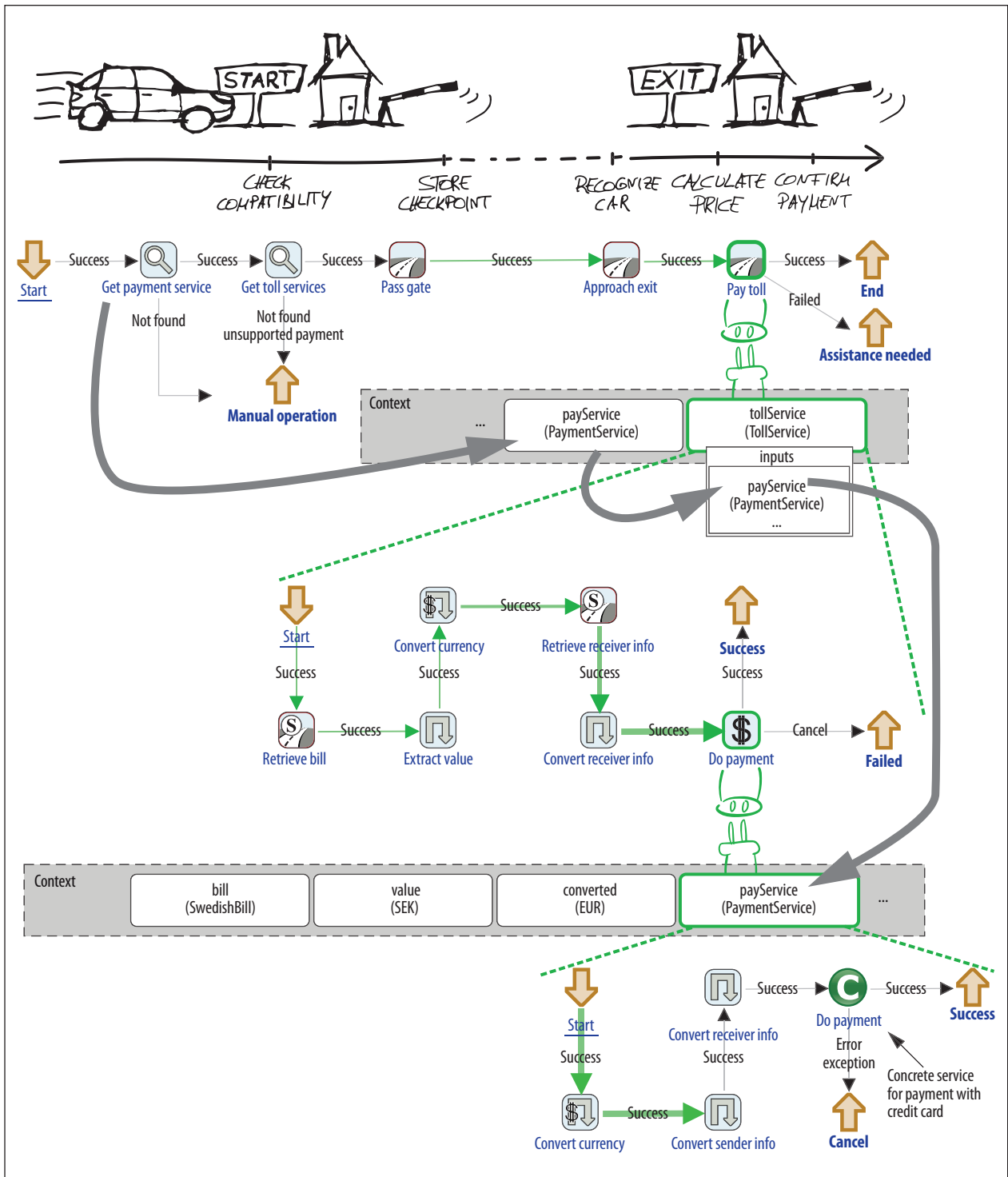


Figure 3. The higher-order business process model of the in-car FlexToll system.

Both activities, *get payment service* and *get toll services*, look in the respective process lists for type-compatible processes that deliver the required functionality. Both are higher-order SIBs, symbolized by the magnifying glass icon. The SLG library includes a selection of toll processes, as

shown in Figure 1. Matching process instances are stored to the corresponding variable (that is, the instantiated SLGs) in the execution context of the car's FlexToll system, represented by the *tollService* or *payService* objects. The activities *pass gate* and *approach exit* are typically toll-system specific, too. However, for ease of

exposition, we focus on *pay toll*, as the other activities would be treated similarly.

Whereas *payService* is a first-order service, *tollService* is second order because it needs the selected *payService* as an input parameter. Thus, *payService* is a procedural parameter, and its flow is depicted by the thick gray arcs in Figure 3. At exit, the *pay toll* activity must also be aware of these bindings. To illustrate the flexibility of this tamed form of higher-order process modeling, assume that a US individual drives to Uppsala, Sweden, with OBU software made by a German company. The middle level of Figure 3 shows a concrete process model for a Swedish toll service, executed by activity *pay toll* in the main process. Its SLG retrieves a bill from the toll station with receiver information (for payment) and the calculated fee, converts this amount from the local currency (Swedish krona, or SEK) to euro (EUR) as FlexToll system-wide valuta. The execution context at this hierarchy level contains the context variables *bill*, *value*, and *converted* and their respective data objects, as well as a payment process instance in the context variable *payService*. Finally, the business activity *do payment* triggers the execution of the payment service previously identified and passed.

All the service processes must match an interface definition that determines the input/output parametrization. This definition includes strict data types for sender, receiver, value, and so on that correspond to the public, application-level concepts used by the processes at their service interface to communicate with the outside world. Internally, however, a concrete payment process converts these data types to the concrete types used by the target payment service, calls the service, and translates the result back to the model-level data types. In our case, the supported value in euro is translated into US dollars (USD), since the driver pays with a US credit card.

This therefore demonstrates the ability to plug in entire process models for a given business activity, as illustrated with *pay toll* for the first hierarchy level and *do payment* for the second, to seamlessly deliver ease of modeling, adaptability, and ultimately flexibility to the entire process. The system does this without needing explicit modeling of all alternatives at design time or explicit management of a large base of variants, and without requiring redeployment of code and restart of ongoing processes for integrating new or varying existing component functionality.

Note the green arrow between the activities *convert currency* and *convert sender info* at the bottom of Figure 3. This snapshot was taken during process execution with the jABC tracer, the built-in SLG interpreter. The green coloring of edges, business activities, and context variables of process instances graphically highlights the concrete execution trace in the process model. The tracer can also serve as a systematic and step-by-step debugging facility in the modeling framework, showing the hierarchy, contexts,

and concrete data values. In this snapshot, the execution is paused in the business activity *convert sender info* within the second hierarchy level (the actual payment process).

BENEFITS AND IMPACT

Higher-order modeling's winning attribute is best demonstrated by noting that the FlexToll process model is totally unaware of which toll service it will execute, and the selected toll service is unaware of which payment services it might execute. All that is necessary is that the chosen services adhere to the required interface, which in our approach is achieved by a strict higher-order type discipline that guarantees executability. This means, for example, that

- any toll or payment service can be added to the FlexToll scenario simply by providing a corresponding type-correct interface, with no changes to the FlexToll OBU; and
- executability is maintained independent of the selected combination of type-correct toll process or payment process instances.

Thus, there is no explosion in model size or number of variants, and the full FlexToll scenario can be maintained elegantly in a “write once, run everywhere” fashion.

Prohibiting recursion enforces a clear separation between algorithmic aspects and variability modeling.

The price to pay is the complexity of a higher-order modeling and execution environment. The looked-up payment process is passed as a parameter to the looked-up toll service, so this example is already third order. More concretely, FlexToll has a process parameter (*tollService*) that has another process parameter (*payService*) as a parameter. We organize these dependencies in a strictly hierarchical fashion: the two relations *is parameter of* and *can be called by* form strict partial orders. Whereas the first preorder restriction is pretty standard, the latter excludes recursion, a fact essential for our notion of *tamed* higher orderedness, which is tailored for achieving transparent and intuitive variability modeling at the process-model level. In our experience, prohibiting recursion enforces a clear separation between algorithmic aspects, which are typically technical in nature and local to specific processes, and variability modeling, which concerns the global interplay between processes, services, and data at the business level. This modeling level benefits most from the ability to treat all these entities alike—as objects,

as parameters, and across multiple hierarchy levels. In fact, most process models do not seem to require recursion, and where the need arose (in algorithmic parts), we safely resolved it at an intraprocess level. Using jABC, such higher-order process models are directly executable, and in fact, their execution can be intuitively visualized, as described earlier.

To maintain the intended transparency and simple plug-and-play intuition, we did not allow recursion at the interprocess level. Our experience with this decision is rather positive. Most process models do not seem to require recursion, and where the need arose (in algorithmic parts) we safely resolved it at an intraprocess level. The future will show whether and then how to loosen this restriction if necessary. Currently, we are considering how to best extend jABC's verification technology to also capture these tamed higher-order process models, in order to produce a comprehensive framework for dealing with product-line management, variability modeling, and self-adaptation technology in a constraint-based fashion.⁵ 

References

1. T. Allweyer, *BPMN 2.0: Introduction to the Standard for Business Process Modeling*, Books on Demand (BoD), 2009.
2. F.J.v.d. Linden, K. Schmid, and E. Rommes, *Software Product Lines in Action: The Best Industrial Practice in Product Line Engineering*, Springer, 2007.
3. J. Pasley, "How BPEL and SOA Are Changing Web Services Development," *IEEE Internet Computing*, vol. 9, no. 3, 2005, pp. 60-67.
4. J. Recker and J. Mendling, "On the Translation between BPMN and BPEL: Conceptual Mismatch between Process Modeling Languages," *18th Int'l Conf. Advanced Information Systems Eng. (CAiSE)*, Proc. Workshops and Doctoral Consortium, Namur Univ. Press, 2006, pp. 521-532.
5. S. Jörges et al., "A Constraint-Based Variability Modeling Framework," *Int'l J. Software Tools for Technology Transfer*, vol. 14, no. 5, 2012, pp. 511-530.
6. A.-L. Lamprecht, S. Naujokat, and I. Schaefer, "Variability Management beyond Feature Models," *Computer*, vol. 46, no. 11, 2013, pp. 48-54.
7. A. Bennaceur et al., "Towards an Architecture for Runtime Interoperability," *Proc. Int'l Conf. Leveraging Applications of Formal Methods, Verification, and Validation (ISoLA)*, vol. 2, LNCS 6416, Springer, 2010, pp. 206-220.
8. I. Schaefer et al., "Software Diversity: State of the Art and Perspectives," *Int'l J. Software Tools for Technology Transfer*, vol. 14, no. 5, 2012, pp. 477-495.
9. T. Margaria and B. Steffen, "Service-Oriented: Conquering Complexity with XMDD," *Conquering Complexity*, M. Hinchey and L. Koyle, eds., Springer, 2012.
10. B. Steffen et al., "Model-Driven Development with the jABC," *Proc. Int'l Haifa Verification Conf. Hardware and Software, Verification, and Testing (HVC 06)*, LNCS 4383, Springer, 2006, pp. 92-108.
11. B.S.T. Margaria, "Business Process Modeling in the jABC: The One-Thing Approach," *Handbook of Research on Business Process Modeling*, Jo Cardoso and W. van der Aalst, eds., IGI Global, 2009, pp. 1-26.
12. M. Müller-Olm, D. Schmidt, and B. Steffen, "Model-Checking: A Tutorial Introduction," *Proc. Int'l Symp. Static Analysis (SAS 99)*, Springer, 1999, pp. 330-354.
13. B. Steffen and T. Margaria, "Metaframe in Practice: Design of Intelligent Network Services," *Correct System Design: Recent Insight and Advances*, LNCS 1710, Springer, 1999, pp. 390-415.
14. C. Seceleanu and I. Crnkovic, "Component Models Made for Reasoning," *Computer*, vol. 46, no. 11, 2013, pp. 40-47.
15. J. Neubauer, B. Steffen, and T. Margaria, "Higher-Order Process Modeling: Product-Lining, Variability Modeling and Beyond," *Electronic Proc. in Theoretical Computer Science*, vol. 129, 2013, pp. 259-283.
16. T. Margaria and B. Steffen, "Second-Order Semantic Web," *Proc. Ann. IEEE/NASA Software Eng. Workshop (SEW 05)*, IEEE CS Press, 2005, pp. 219-227.

Johannes Neubauer is a PhD student in the Department of Computer Science at Technische Universität Dortmund, Germany. His research interests include model-driven development, variability modeling, programming in the large, service-oriented computing, and automata learning. Contact him at johannes.neubauer@cs.tu-dortmund.de.

Bernhard Steffen is the chair of programming systems in the Department of Computer Science at Technische Universität Dortmund, Germany. His research interests include program analysis and optimization, model-based analysis of distributed systems, automata learning, and service-oriented computing. Steffen received a PhD in computer science from Christian-Albrechts-Universität zu Kiel, Germany. He is a member of IEEE, ACM, International Federation for information Processing, the German Informatics Society, and European Association of Software Science and Technology. Contact him at steffen@cs.tu-dortmund.de.



 Selected CS articles and columns are available for free at <http://ComputingNow.computer.org>.